

PA1 – 3D Shapes in WebGL

Student ID: 241482

Assigned solid: Wedge

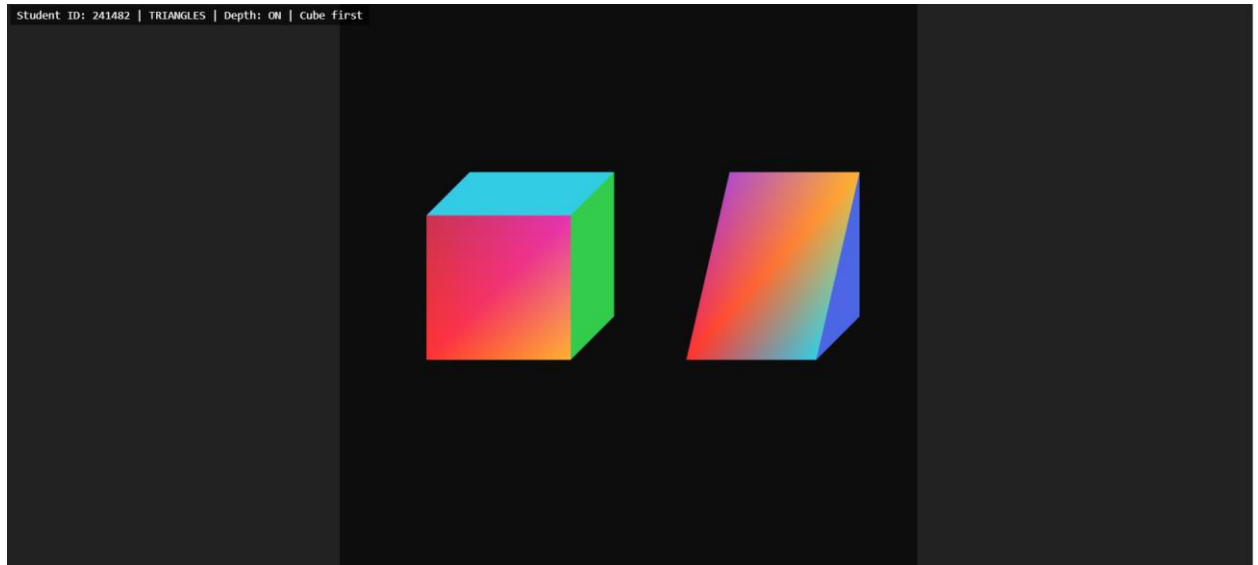
Depth offset: (+0.15, +0.15)

E1 – Depth Test vs Draw Order

Prediction: I expected that with the depth test enabled, the closer surfaces would appear on top of the farther ones regardless of the order they are stored in the array.

Result: On the screen, the cube and wedge are displayed as solid shapes. For the cube, the front, top, and right faces are visible. The wedge is located to the right of the cube, and its visible surfaces are also shown with depth taken into account.

Explanation: With `DEPTH_TEST` enabled, WebGL uses the depth buffer and compares fragment depths. Therefore, closer surfaces cover farther ones, and the result is not determined solely by the order of drawing the triangles.



Prediction: I expected that after turning off depth testing, the drawing order would affect which surfaces appear on top of others.

Result: After turning off `DEPTH_TEST`, the way the faces were displayed changed, meaning some surfaces that should be behind ended up covering closer surfaces. This happened because WebGL no longer compares fragment depth.

Explanation: With depth testing off, the depth buffer isn't used to determine the nearest surface. So when triangles overlap, the result depends on the order they're drawn. Faces drawn later can appear on top of faces that are geometrically closer to the viewer.



Prediction: I expected that after pressing S, the drawing order of objects would change, meaning the wedge would be drawn first and then the cube.

Result: After switching the draw order, it barely looked any different. The cube and wedge still look the same, even though the wedge is now drawn first.

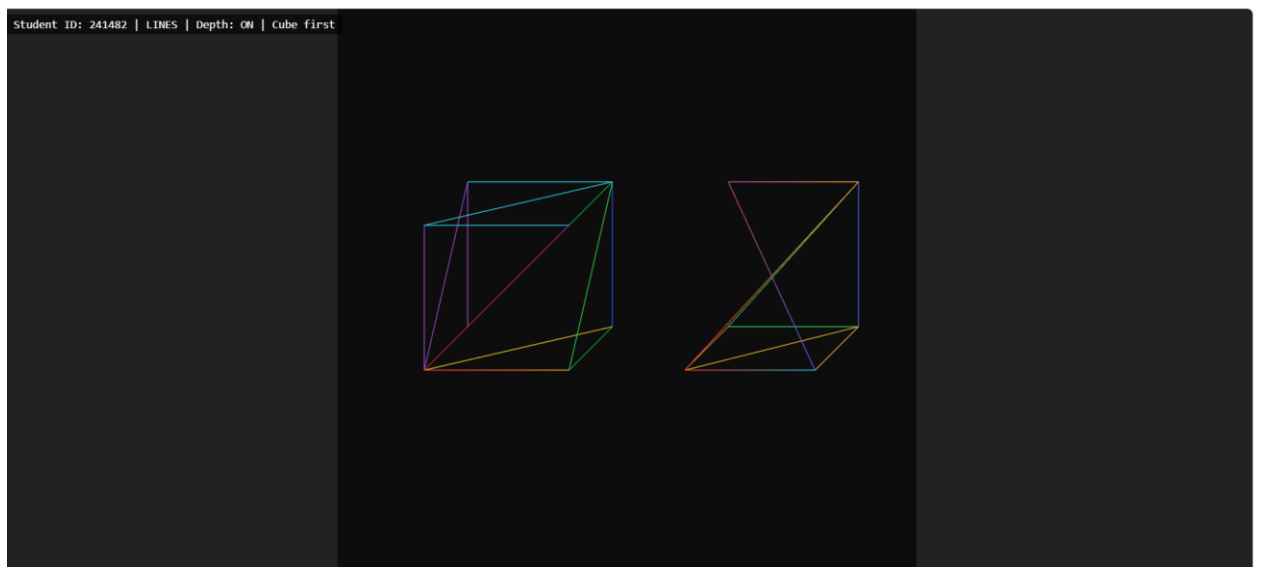
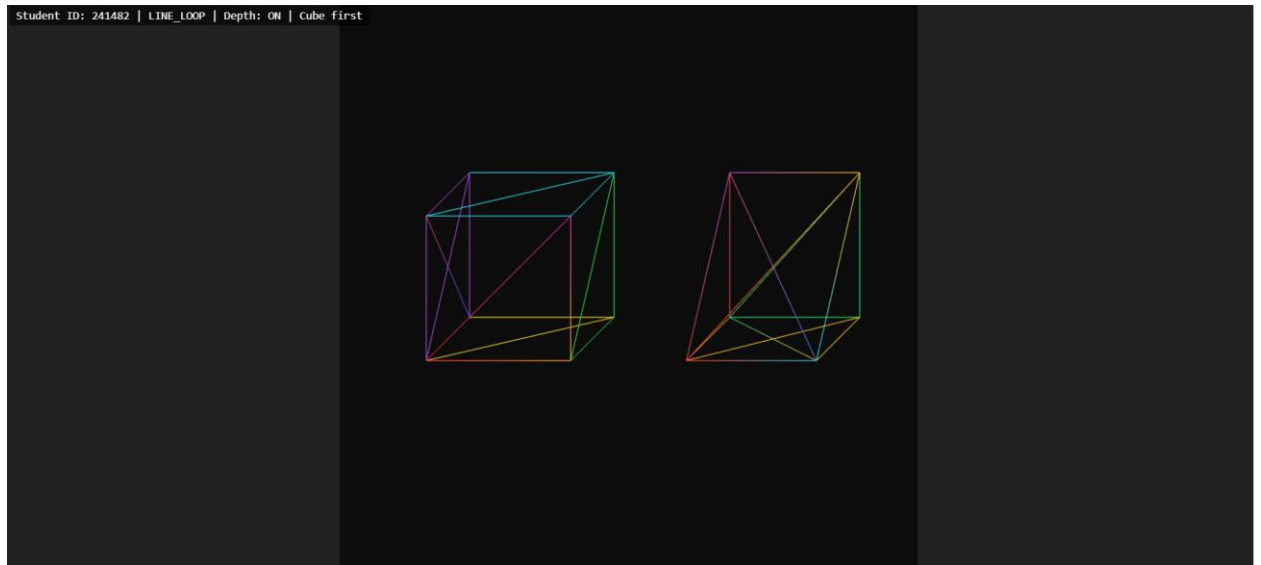
Explanation: This happens because the cube and wedge are located in different parts of the canvas and don't overlap. So changing the order of the two `gl.drawArrays()` calls doesn't affect the final image. The effect of draw order is only noticeable when objects occupy the same pixels on the screen.

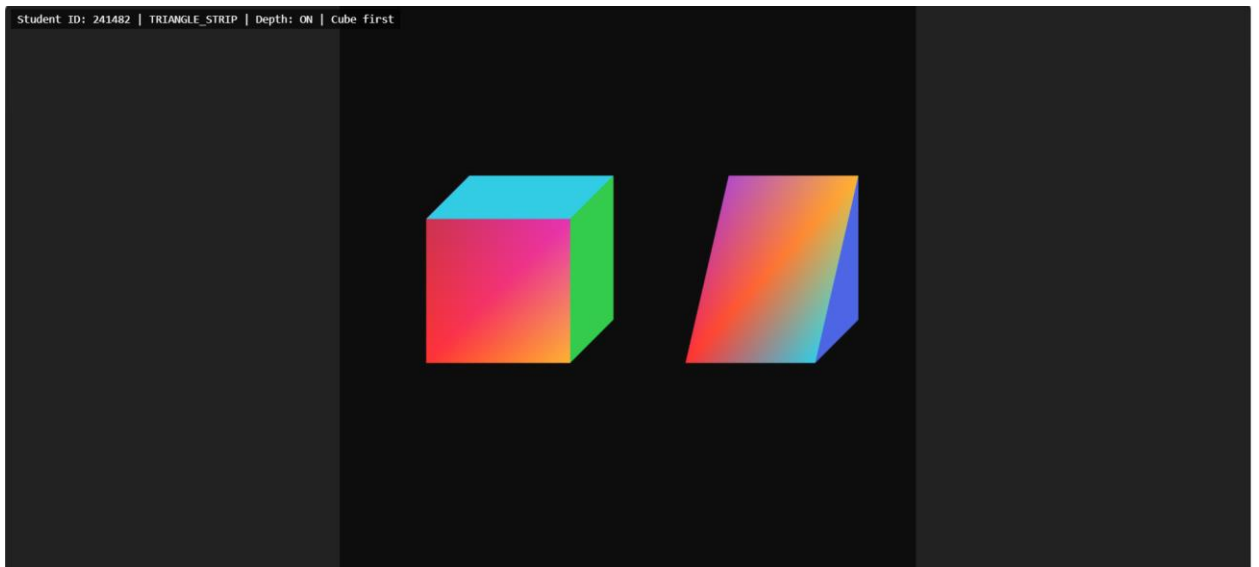


Prediction: I expected the same vertices to look different depending on the chosen drawing mode because the vertex array was prepared for the TRIANGLES mode.

Result: In LINE_LOOP mode, extra diagonal lines and connections between vertices appeared. In LINES mode, only some lines were shown, so the shape didn't look like a full wireframe. In TRIANGLE_STRIP mode, the triangles came out wrong.

Explanation: The vertex array was made for independent triangles. LINE_LOOP connects consecutive vertices into a single closed line, so extra connections appear. LINES treats vertices in pairs, so only separate segments are displayed. TRIANGLE_STRIP uses neighboring vertices to build the next triangles, so data prepared for TRIANGLES is interpreted incorrectly.





E5 : Prediction: I expected that after changing the coordinate of one vertex of the cube from $z = -0.5$ to $z = 1.2$, this vertex would go beyond the allowed clip space and part of the triangle would be clipped.

Result: After changing z , the appearance of the cube didn't change much. This is because only one vertex of one triangle was changed, while the neighboring triangle of the front face remained unchanged.

Explanation: In WebGL, after transformation, the visible area is limited by the clip space. The value $z = 1.2$ is beyond the allowed limit of $+1$, so the part of the primitive that is beyond this boundary gets clipped. WebGL, however, keeps the visible part of the triangle that remains inside the clip volume.



Prediction: I expected that after changing the coordinate of one vertex of the cube to $x = 1.4$, this vertex would go outside the clip space along the X axis, so part of the corresponding triangle would be clipped.

Result: After changing x , the appearance of the cube didn't change much at all. Most of the cube remained visible because only one vertex of a single triangle was changed, and the other vertices stayed within the allowed area.

Explanation: In WebGL, coordinates after transformation should be within the clip space. The value $x = 1.4$ is beyond the right boundary of $+1$, so the part of the triangle that goes past this boundary gets clipped. Only the part of the primitive that is inside the clip volume remains visible.



E6 : The total number of vertices in my scene is 60: the cube has 36 vertices, and the wedge has 24 vertices.

The position buffer stores three Float32 values for each vertex: x , y , and z . One Float32 takes up 4 bytes, so the size of the position buffer is:

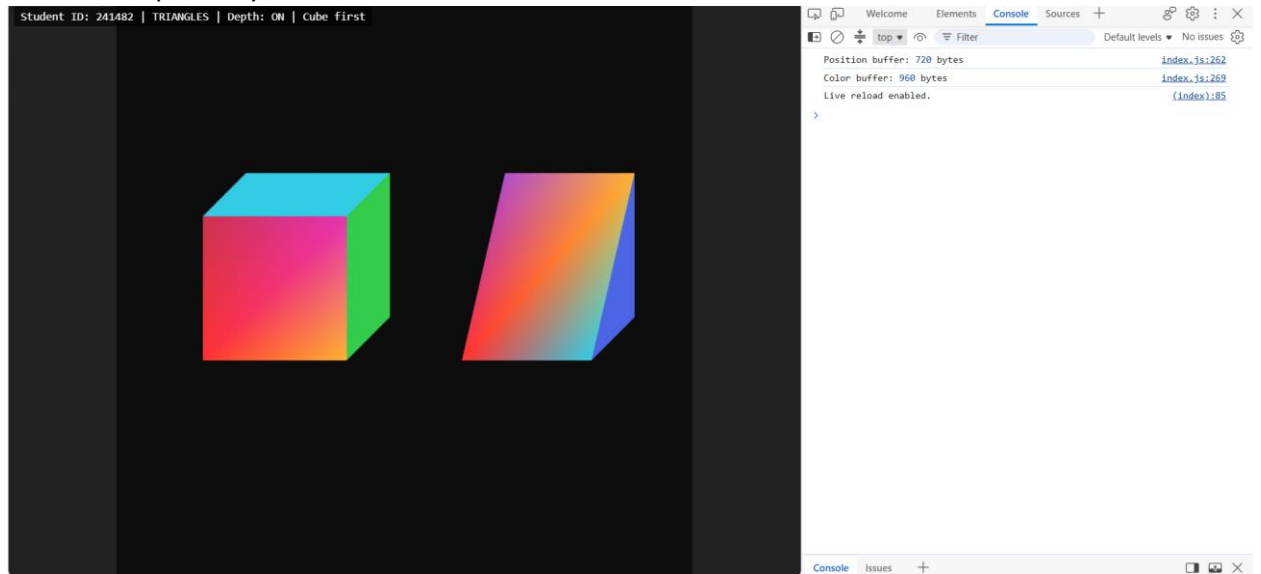
$$60 \times 3 \times 4 = 720 \text{ bytes.}$$

The color buffer stores four Float32 values for each vertex: r , g , b , and a . So its size is:

$$60 \times 4 \times 4 = 960 \text{ bytes.}$$

I checked these values using `gl.getBufferParameter(gl.ARRAY_BUFFER, gl.BUFFER_SIZE)`. The Console showed 720 bytes for the position buffer and 960 bytes for the color buffer, which matches the

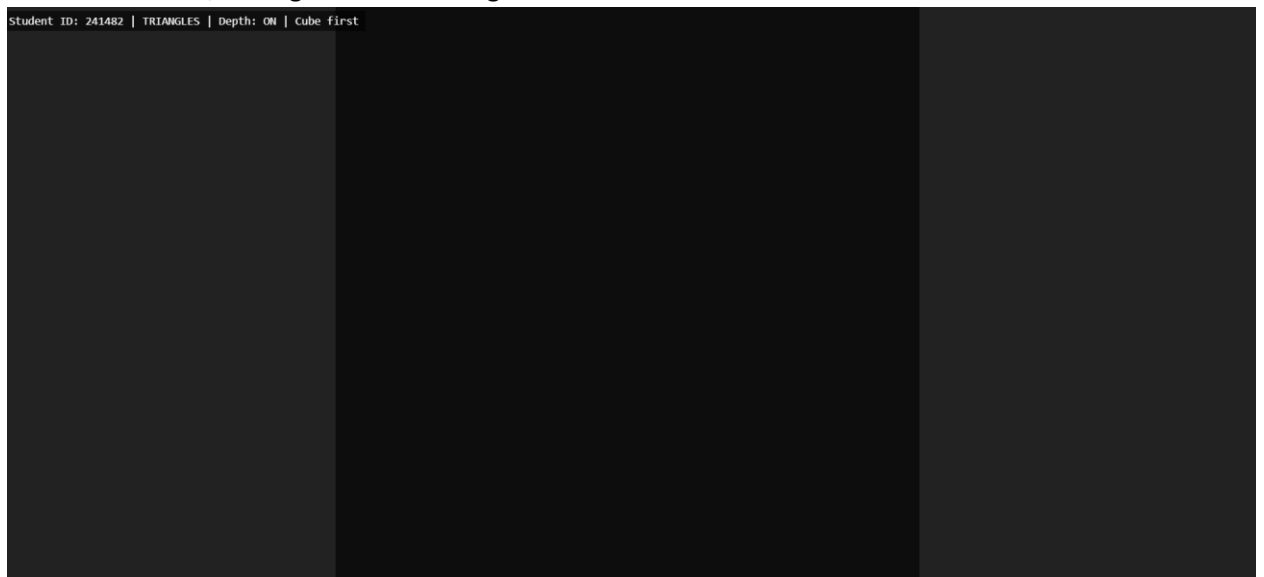
calculations perfectly.



E2 : Prediction: I expected that if I replace `gl.LEQUAL` with `gl.GREATER` but keep the default depth buffer clear value at 1.0, most or all fragments wouldn't pass the depth test.

Result: After setting `gl.GREATER`, objects stopped displaying correctly.

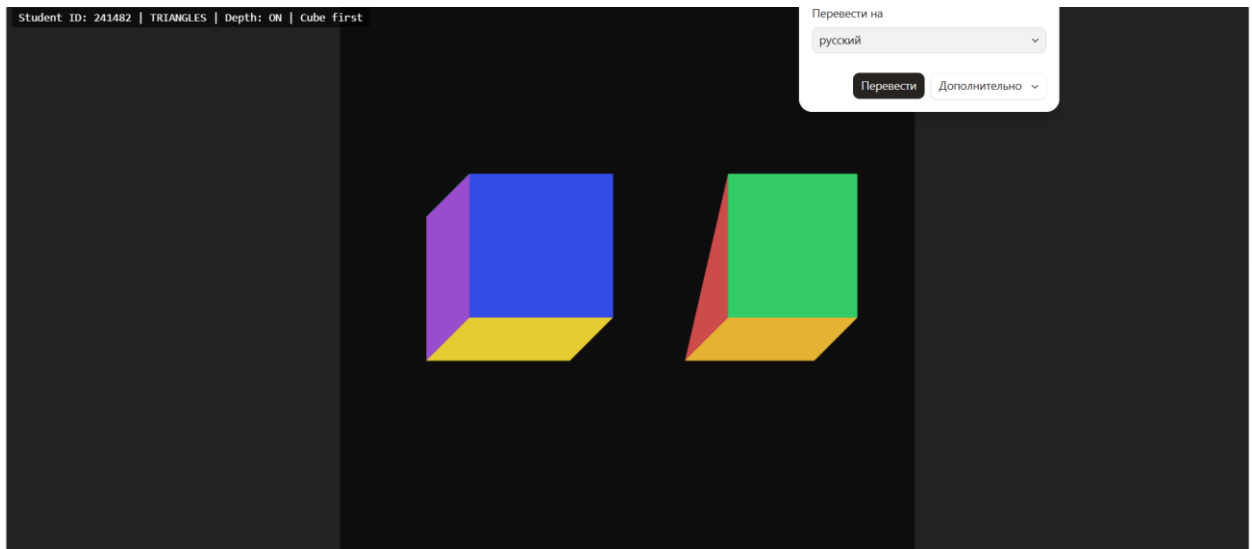
Explanation: By default, the depth buffer is cleared to 1.0. With the `GREATER` function, a new fragment has to have a depth value greater than the one already stored. But valid depth values are roughly between 0 and 1, so fragments can't be greater than 1.0 and fail the test.



E2 : Prediction :I expected that after adding `gl.clearDepth(0.0)` along with `gl.depthFunc(gl.GREATER)`, the objects would become visible again, because the fragment depth values would be greater than the initial depth buffer value of 0.0.

Result: After setting `gl.clearDepth(0.0)`, both shapes appeared again. But the visible surfaces changed: now the surfaces that were farther back with the standard `gl.LEQUAL` came to the foreground.

Explanation: With the standard `gl.LEQUAL`, smaller depth values pass the depth test, so surfaces with negative z appear closer. Switching to `gl.GREATER` changes this: now fragments with greater depth pass. Clearing the depth buffer to 0.0 allows such fragments to pass the test, so more positive z values start appearing over the negative ones.



E4 : Prediction: I expected that if aPosition was linked to the color buffer instead of the position buffer, WebGL would start using color values as vertex coordinates, so the scene's geometry would be displayed incorrectly.

Result: After incorrectly binding the buffer, the shape of objects became distorted / the scene stopped displaying correctly.

Explanation: `gl.vertexAttribPointer()` remembers the `ARRAY_BUFFER` that was bound at the time the function was called. So aPosition started reading data from the color buffer. WebGL doesn't report an error because the buffer itself exists and contains valid numeric data. The error is logical: WebGL doesn't know that this data was meant for color, not for position.



Development Log

During development, I found a bug in the wedge geometry: at first, the shape looked like a triangular prism rather than a ramp. I noticed the issue while visually inspecting the scene in TRIANGLES and LINE_LOOP modes. I adjusted the positions of the wedge vertices so that the slope rose toward the back of the shape, and I rechecked the result via Live Server. The fix was saved in commit **e58e39f** — “Complete wedge geometry and WebGL controls”.